C PROGRAMMING ASSIGNMENT 1

DESIGN, IMPLEMENT AND VALIDATE A RELIABLE BILLING PROGRAM FOR A RURAL ELECTRICITY CO-OPERATIVE

Student Name: Sri Varshini K

Register Number: 192512328

Programming Language: C

Compiler: GCC 6.3.0

Submission Evidence: Tasks 01–12

S.No	CONTENT
1	Introduction and Problem Statement
2	Objectives and Problem Analysis
3	Data Types and Tariff Constants
4	Algorithms and Pseudocode
5	UML / Module Diagrams
6	Implementation Evidence: Tasks 01–12
7	Test Plan and Results
8	Compilation Stages
9	Complexity Analysis
10	Numeric Accuracy and Overflow
11	Smart Meter Design Challenge
12	Limitations and Future Improvements
13	Conclusion
14	References
Appendix A. Screenshot Evidence	

1. Introduction and Problem Statement

Electricity billing requires accurate consumption calculations, correct tariff application, and reliable handling of exceptional conditions such as meter rollover and invalid input. This assignment applies C programming concepts to a rural electricity co-operative billing scenario.

Tariff rules used in the assignment:

Component	Rule
Fixed charge	₹50.00 per bill
First 100 units	No energy charge
Units 101–200	₹2.25 per unit
Units above 200	₹4.50 per unit
Meter register	16-bit unsigned range 0–65535; rollover to 0

The task folders contain separate demonstrations rather than one integrated production application. The smart meter design in Task 12 is a proposal; it is not a complete implementation of telemetry, persistence, or runtime tariff loading.

2. Objectives and Problem Analysis

• Calculate electricity charges according to the given slab structure.

• Represent monetary values as integer paise to avoid binary floating-point errors in billing arithmetic.

• Calculate meter consumption and demonstrate rollover handling.

• Demonstrate data types, macros, conditional compilation, input validation, type conversion, and GCC compilation stages.

• Test ordinary, boundary, rollover, and invalid-input scenarios.

• Describe a smart meter design with 15-minute readings and a 20% peak-hour uplift from 6 PM to 10 PM.

2.1 Inputs, Processing, Outputs

Category	Details
Inputs	Previous/current meter readings; or directly entered consumption units; smart-meter proposal additionally uses timestamps and tariff settings.
Processing	Validate input; compute consumption; account for rollover; calculate energy charge by slab; add ₹50 fixed charge.
Outputs	Consumption units, bill in rupees and paise, data-type/precision demonstrations, and error messages for invalid input.

2.2 Alternative Representation

Integer representation permits direct arithmetic and requires O(1) time and O(1) auxiliary space for a single fixed-width reading calculation. String representation requires validation and conversion before arithmetic; conversion takes O(d) time for d digits.

3. Data Types and Tariff Constants

Type / constant	Role in the project	Important note
unsigned short / meter_t	Meter register value	The C standard does not guarantee 16 bits; verify USHRT_MAX == 65535 on the target platform.
unsigned int	Non-negative consumption	Used for consumption output and combined tests.
long / long long	Bill paise and accumulated totals	Choose and validate a type large enough for the maximum supported consumption and totals.
int	Entered units and ordinary integer examples	Negative values must be rejected where units cannot be negative.
float / double	Floating-point demonstration	Not preferred for exact monetary arithmetic.
RATE2_PAISE = 225	₹2.25 per unit	Integer paise.
RATE3_PAISE = 450	₹4.50 per unit	Integer paise.
FIXED_CHARGE_PAISE = 5000	₹50.00 fixed charge	Integer paise.

3.1 Billing Formula

```
If U <= 100: energy_paise = 0
If 100 < U <= 200: energy_paise = (U - 100) * 225
If U > 200: energy_paise = (100 * 225) + ((U - 200) * 450)
total_paise = 5000 + energy_paise
```

The formula above matches the implemented slab calculations. Input and arithmetic range checks are required before using this formula for unrestricted values.

4. Algorithms and Pseudocode

4.1 Meter Consumption and Rollover

```
INPUT previous, current
IF either reading is outside 0..65535:
   REPORT invalid reading
ELSE IF current >= previous:
   units = current - previous
ELSE:
   units = (65536 - previous) + current
OUTPUT units
```

This pseudocode describes a 16-bit register. The implementation must verify that the selected C type is exactly suitable for this range.

4.2 Billing

```
INPUT units
IF input is not a valid whole number OR units < 0:
   REPORT input error
ELSE:
   energy_paise = 0
   IF units > 200:
      energy_paise = 100 * 225 + (units - 200) * 450
   ELSE IF units > 100:
      energy_paise = (units - 100) * 225
   total_paise = 5000 + energy_paise
   DISPLAY total_paise as rupees and two-digit paise
```

4.3 Smart Meter Processing Proposal

```
LOAD and validate tariff configuration
FOR each timestamped reading:
   validate meter id, timestamp, and reading
   identify whether timestamp is in 18:00–22:00 peak window
   select normal tariff or 1.20 × normal tariff
   calculate interval consumption according to the defined meter policy
   accumulate energy charge using integer paise
STORE readings and billing audit data
ADD fixed charge and DISPLAY bill
```

The uploaded Task 12 notes propose this workflow. They do not include a compiled smart-meter implementation or actual runtime configuration parser.

5. UML / Module Diagrams

Figure 1. Simple use-case diagram for the billing application.

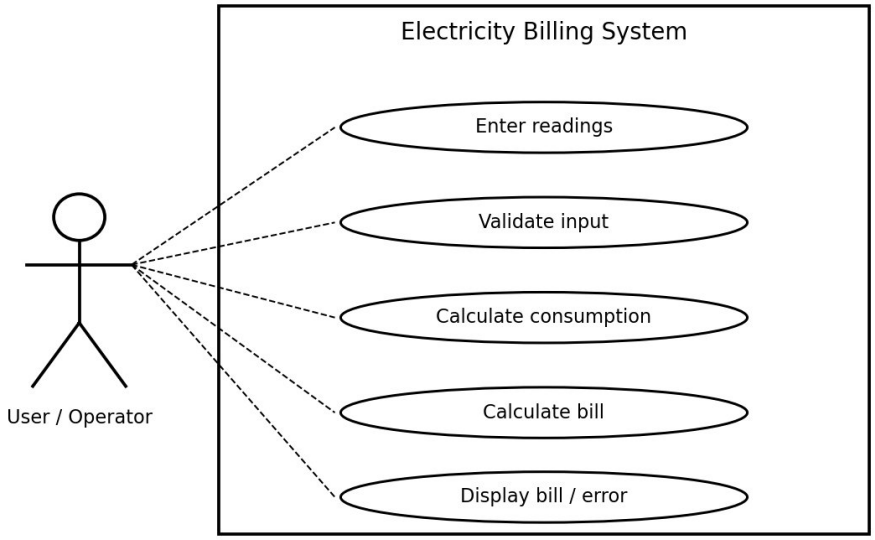
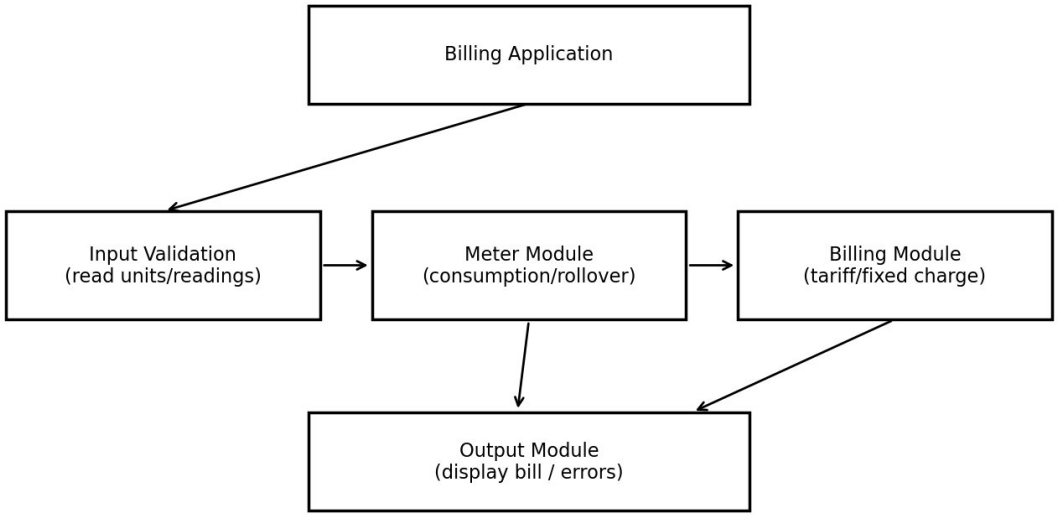


Figure 2. Simple module diagram showing a logical decomposition of the billing application. This is a module-level design diagram, not a claim that these are implemented as separate C classes.



6. Implementation Evidence: Tasks 01–12

Task 01: Problem analysis

analysis.txt records inputs, processing, outputs, alternative representation, and O(1) complexity.

```
Task 01 – Problem Analysis

Problem: Design a reliable electricity billing program for a rural electricity co-operative.

1. Inputs
- Previous meter reading.
- Current meter reading.
- Meter readings are stored in a 16-bit unsigned register ranging from 0 to 65535.
- The meter register can roll over from 65535 to 0.

2. Processing
- Calculate electricity consumption from the previous and current readings.
- Handle meter rollover correctly.
- Apply the billing rules:
  - Fixed charge: ₹50.
  - First 100 units: free.
  - Units 101–200: ₹2.25 per unit.
  - Units above 200: ₹4.50 per unit.
- Calculate the final bill accurately to the nearest paisa.
- Validate user input and handle invalid readings.

3. Outputs
- Electricity units consumed.
- Fixed charge.
- Energy charge.
- Total bill in rupees and paise.
```

```
- Error message for invalid input.

4. Alternative representation of meter readings

Meter readings can be represented as integers or strings.

- Integer representation:** Uses numeric values directly and supports arithmetic operations efficiently.
- String representation:** Stores digi
```

Task 02: Meter type and rollover

main.c defines meter_t as unsigned short and tests normal consumption 1200→1530 and rollover 65400→120.

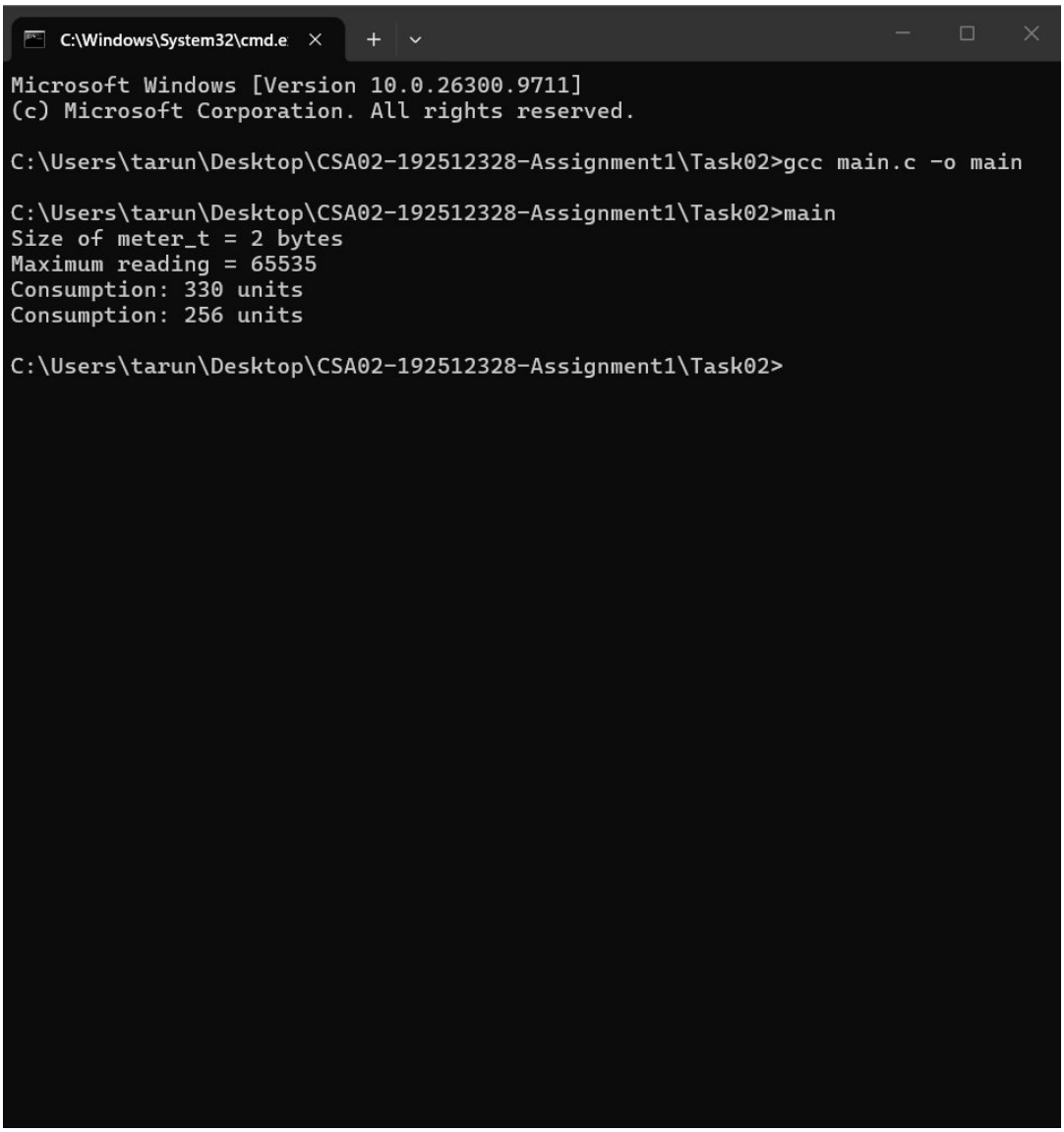
```
#include <stdio.h>
#include <limits.h>

typedef unsigned short meter_t;

unsigned int consumption(meter_t previous, meter_t current)
{
   return (meter_t)(current - previous);
}

int main(void)
{
   printf("Size of meter_t = %zu bytes\n", sizeof(meter_t));
... (see full file in Task02 folder)
```

Evidence screenshot for Task 02:



Task 03: Tariff calculation with constants

main.c defines slab thresholds, rates in paise, fixed charge, and compute_bill().

```
#include <stdio.h>

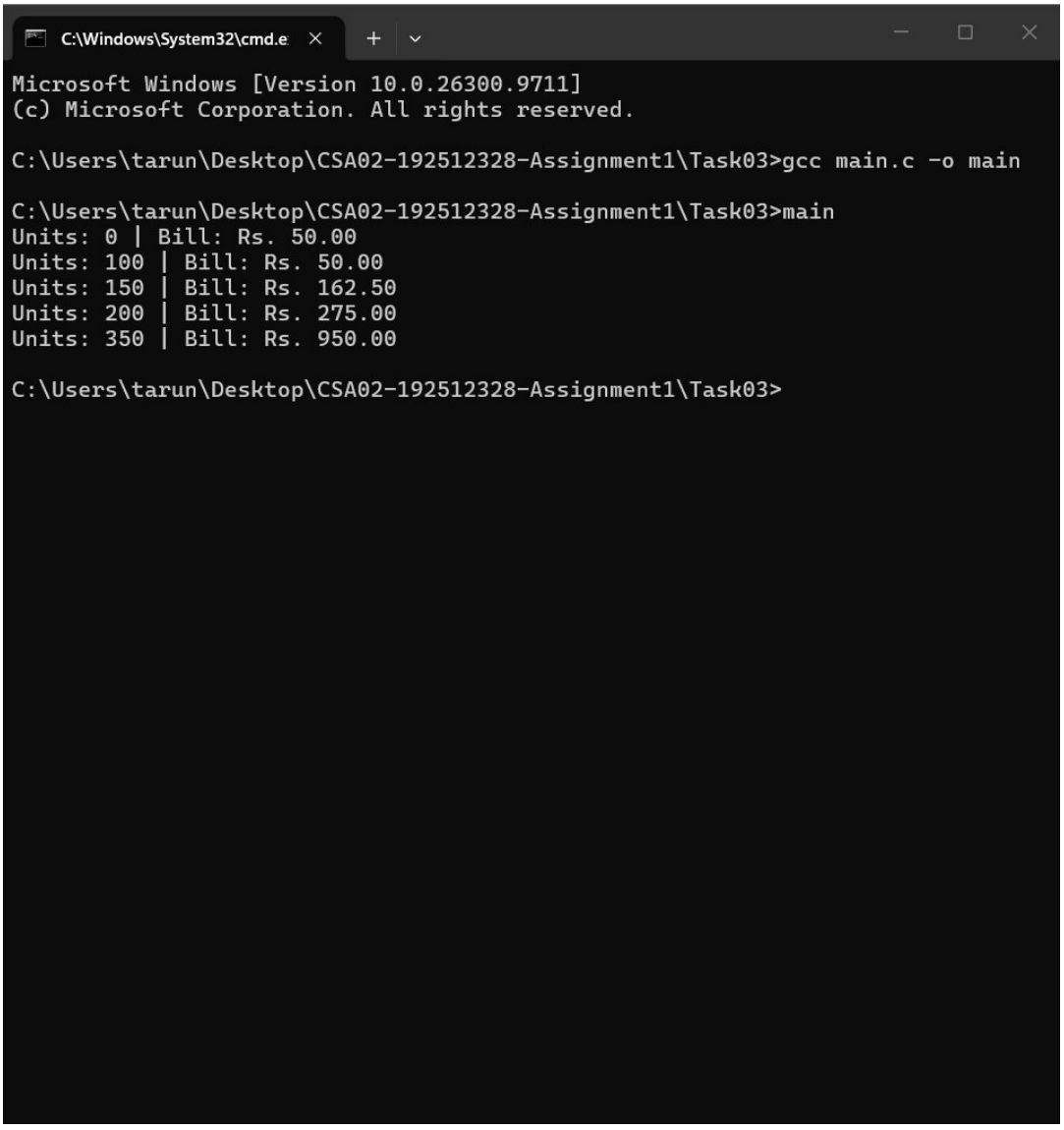
#define SLAB1_UNITS 100
#define SLAB2_UNITS 200

#define RATE1_PAISE 0
#define RATE2_PAISE 225
#define RATE3_PAISE 450

#define FIXED_CHARGE_PAISE 5000L

long compute_bill(int units)
{
... (see full file in Task03 folder)
```

Evidence screenshot for Task 03:



Task 04: Macro safety

main.c contrasts SQUARE_BAD(x) with parenthesized SQUARE(x) and a static inline square() function.

```
#include <stdio.h>

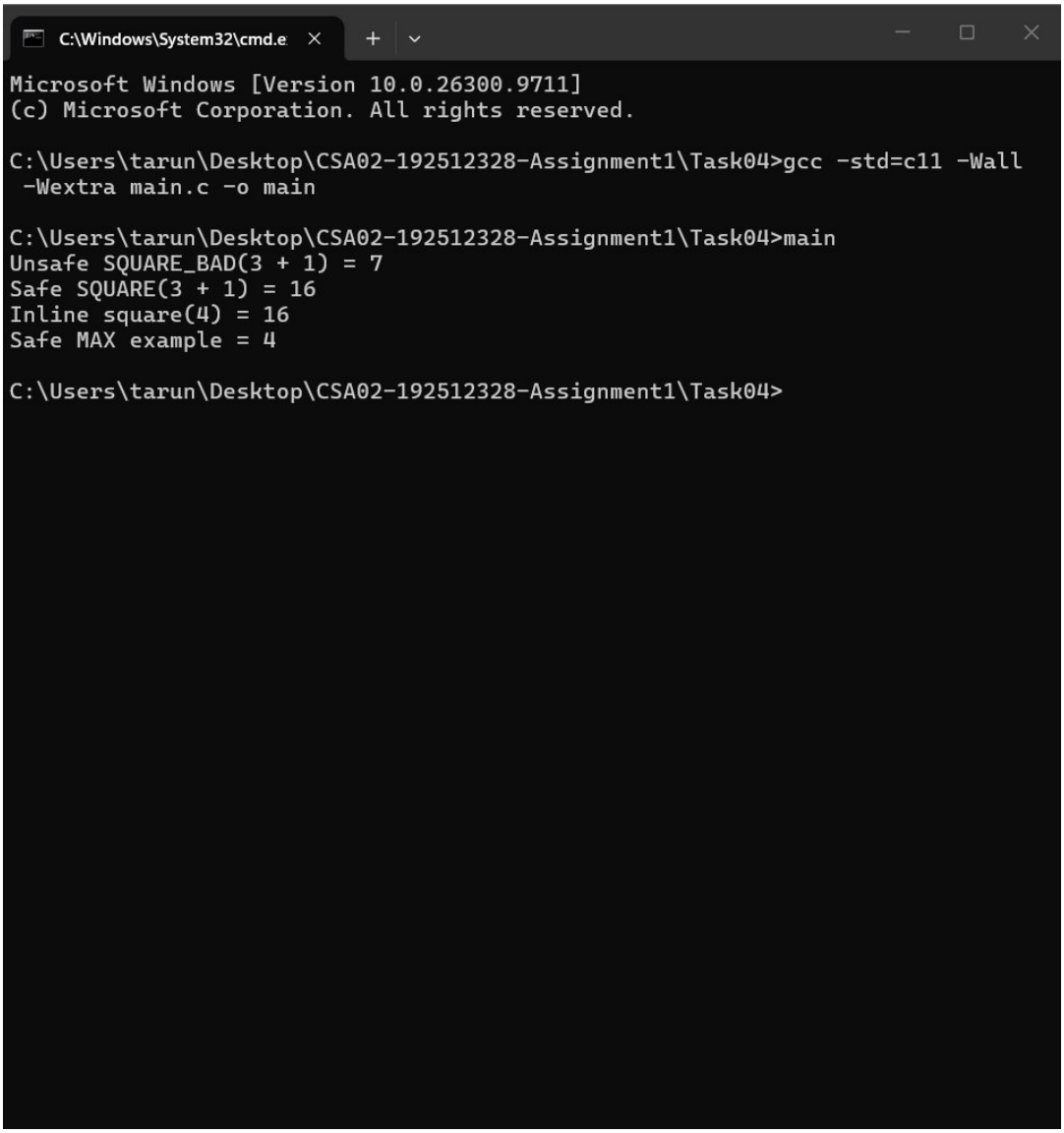
/* Unsafe macro */
#define SQUARE_BAD(x) x * x

/* Safer macro with parentheses */
#define SQUARE(x) ((x) * (x))

/* Safer alternative using an inline function */
static inline int square(int x)
{
    return x * x;
```

```
}
... (see full file in Task04 folder)
```

Evidence screenshot for Task 04:



Task 05: Include guard and conditional compilation

main.c demonstrates an include-guard pattern and optional DEBUG output.

```
#include <stdio.h>

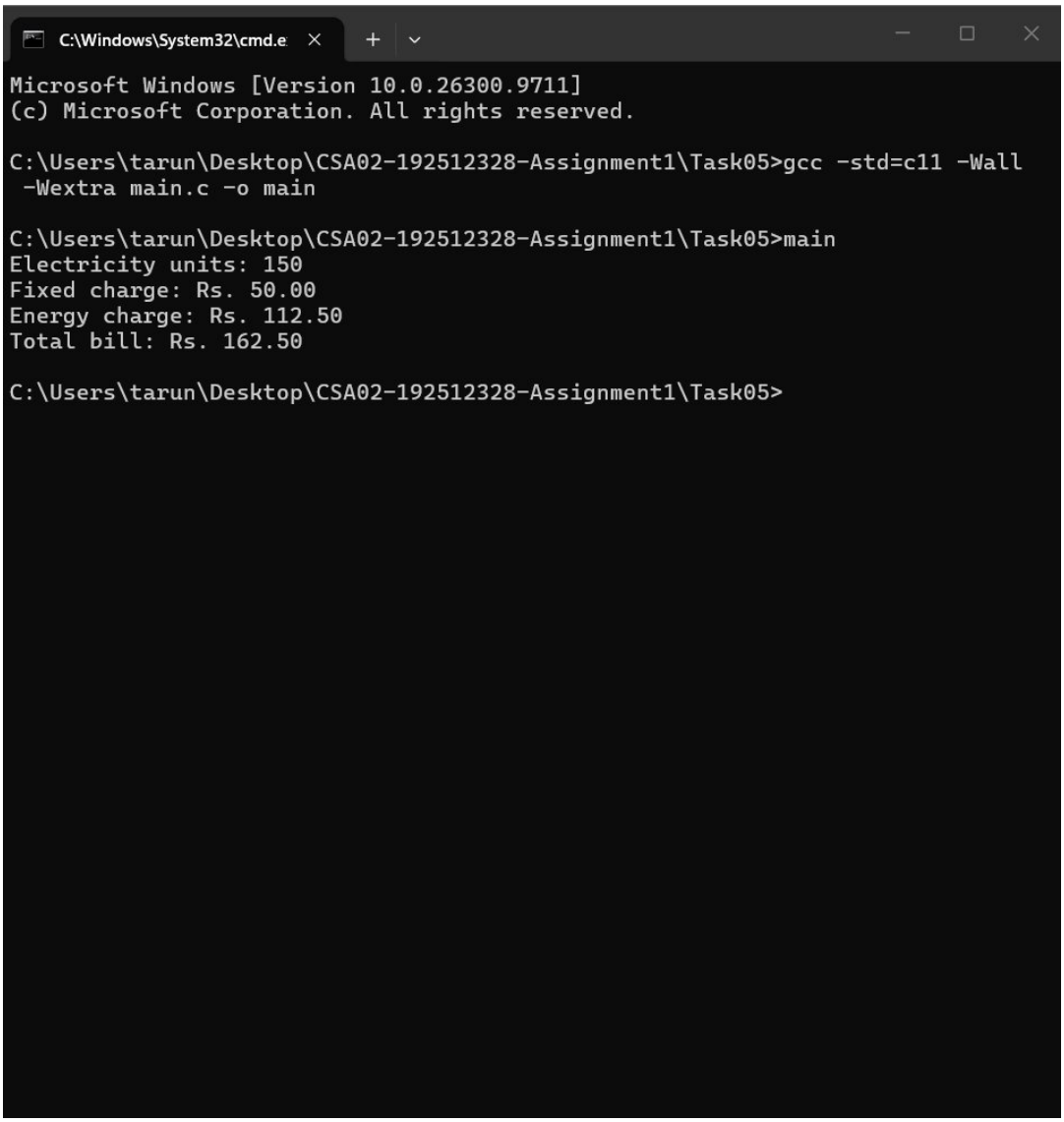
/* Include guard demonstration */
#ifndef BILLING_CONFIG_H
#define BILLING_CONFIG_H

#define FIXED_CHARGE 50
#define RATE_PER_UNIT 2.25

#endif
```

```
int main(void)
{
... (see full file in Task05 folder)
```

Evidence screenshot for Task 05:



Task 06: Input validation

main.c checks scanf() return value and rejects negative units.

```
#include <stdio.h>

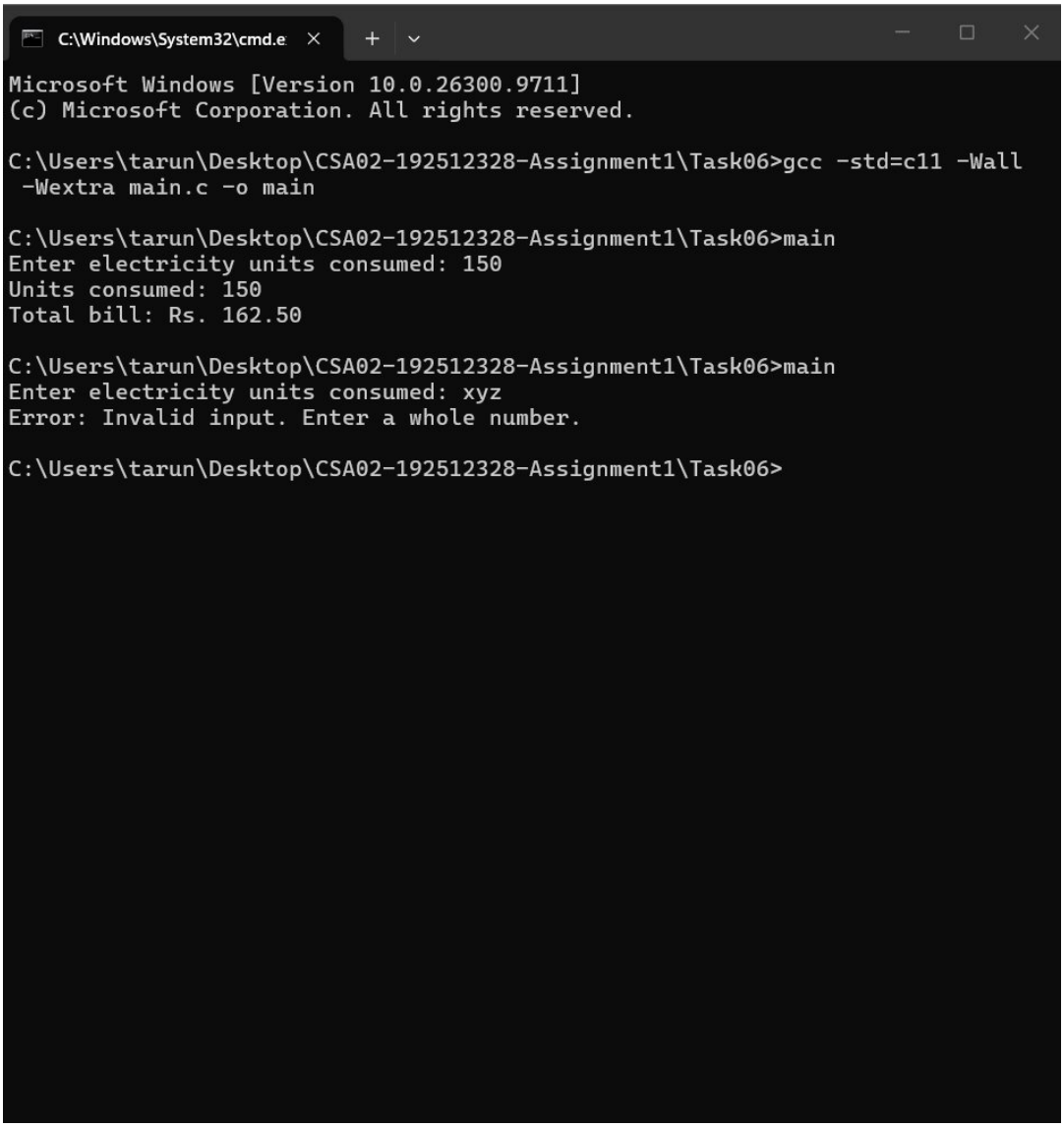
int main(void)
{
   int units;
   int result;

   printf("Enter electricity units consumed: ");
```

```
    result = scanf("%d", &units);

    if (result != 1) {
        printf("Error: Invalid input. Enter a whole number.\n");
... (see full file in Task06 folder)
```

Evidence screenshot for Task 06:



Task 07: Type conversion and precision

main.c demonstrates integer division, double conversion, truncation, integer paise, and double display.

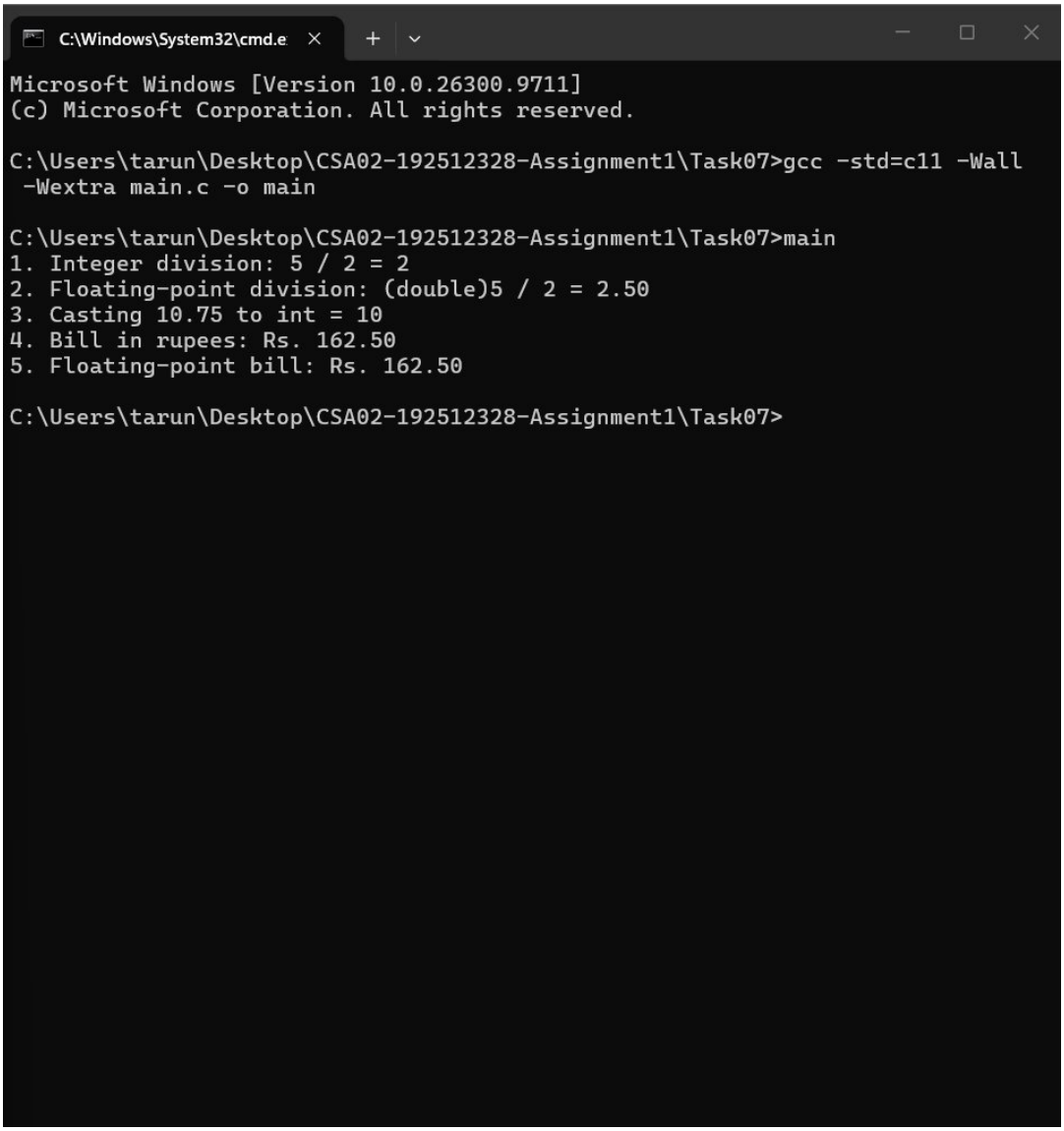
```
#include <stdio.h>

int main(void)
{
    int a = 5;
    int b = 2;
```

```
    /* Example 1: Integer division */
    printf("1. Integer division: 5 / 2 = %d\n", a / b);

    /* Example 2: Explicit type conversion */
    printf("2. Floating-point division: (double)5 / 2 = %.2f\n",
           (double)a / b);
... (see full file in Task07 folder)
```

Evidence screenshot for Task 07:



Task 08: Compilation stages

The folder includes main.c, main.i, main.s, main.o, main.exe, and output.png.

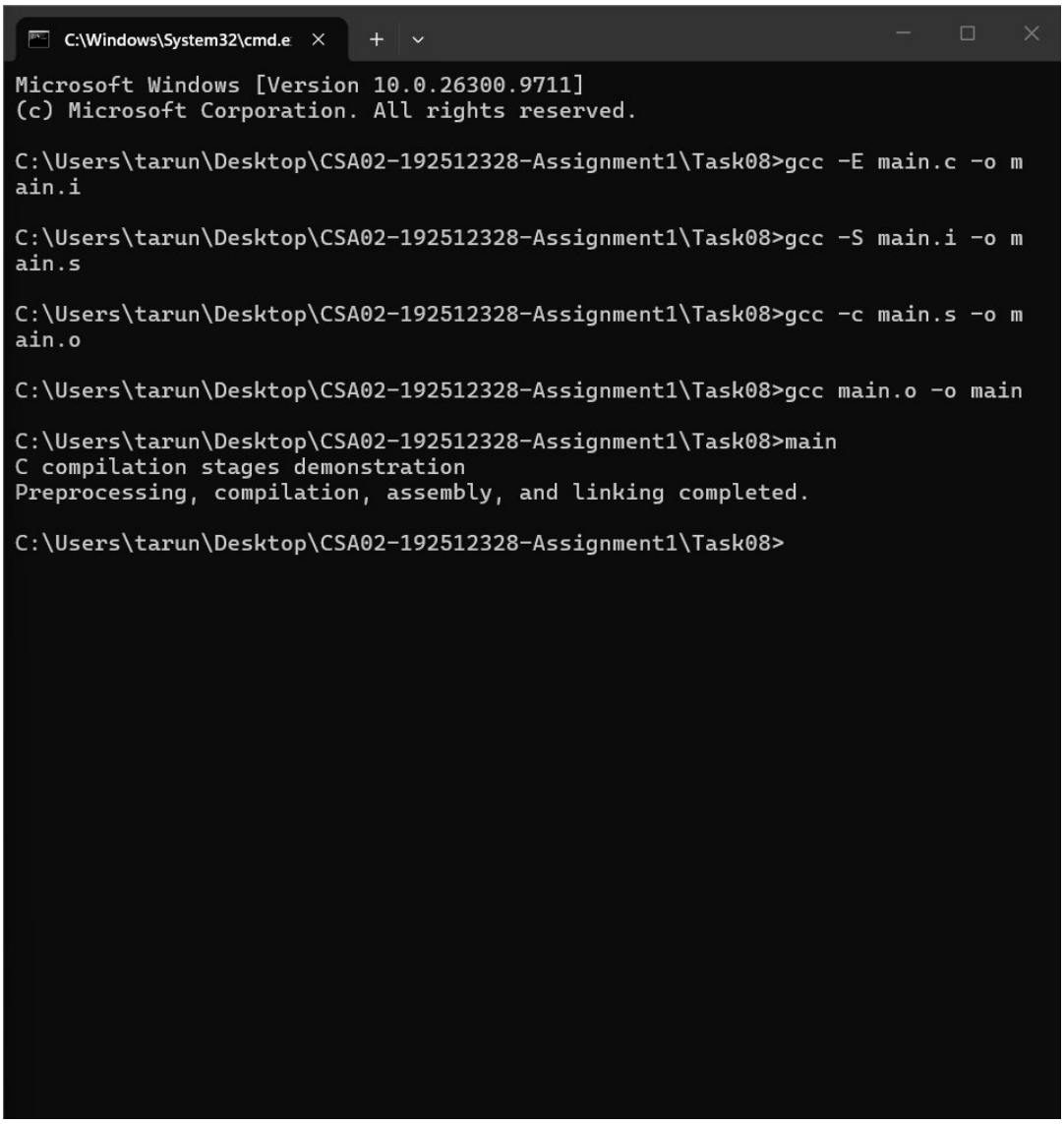
```
#include <stdio.h>

int main(void)
{
```

```
    printf("C compilation stages demonstration\n");
    printf("Preprocessing, compilation, assembly, and linking completed.\n");

    return 0;
}
```

Evidence screenshot for Task 08:



Task 09: Combined meter and billing tests

main.c combines rollover examples with tariff boundary cases.

```
#include <stdio.h>
#include <limits.h>

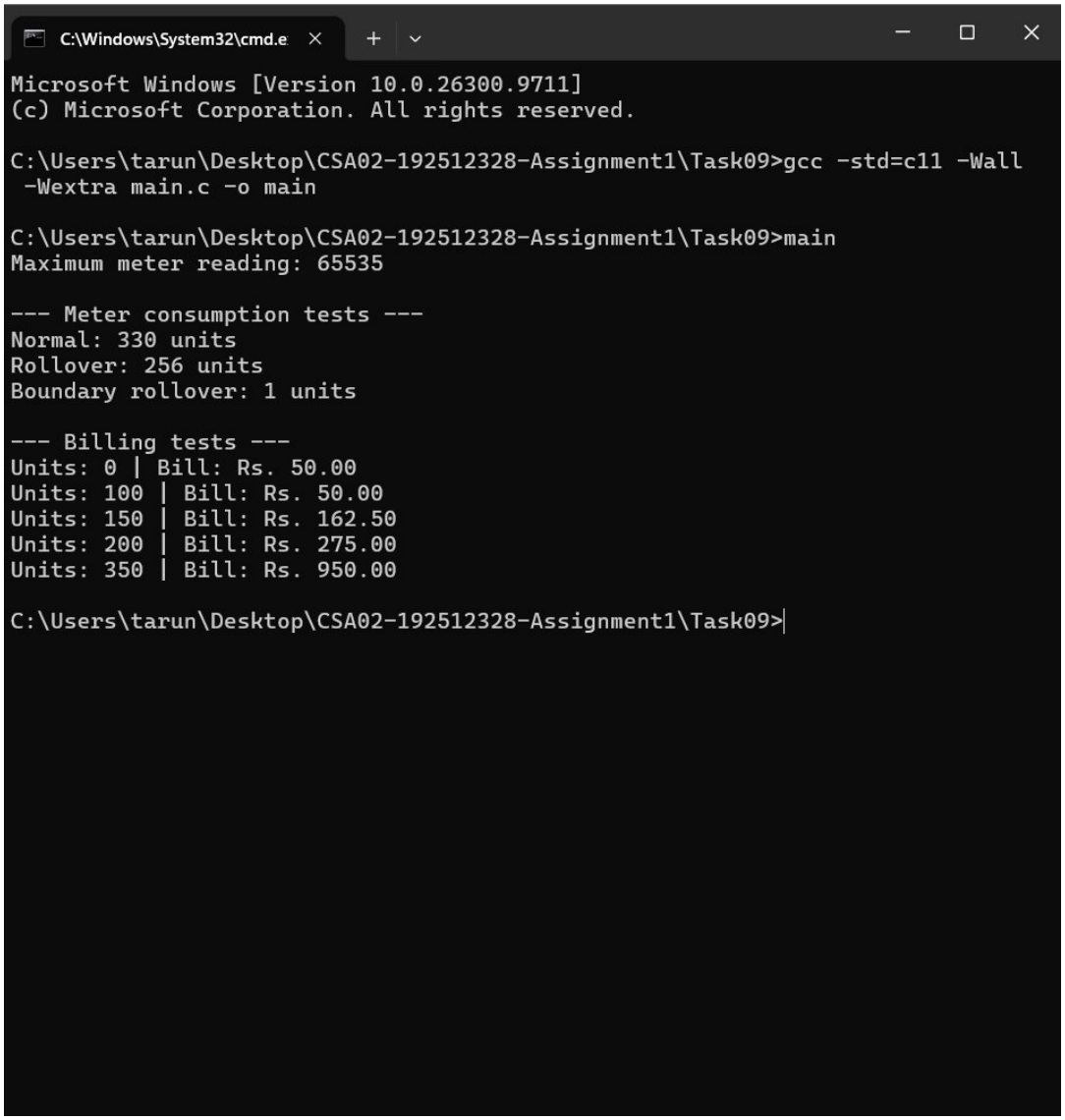
#define SLAB1_UNITS 100
#define SLAB2_UNITS 200
#define RATE2_PAISE 225L
#define RATE3_PAISE 450L
```

```
#define FIXED_CHARGE_PAISE 5000L

typedef unsigned short meter_t;

unsigned int consumption(meter_t previous, meter_t current)
{
... (see full file in Task09 folder)
```

Evidence screenshot for Task 09:



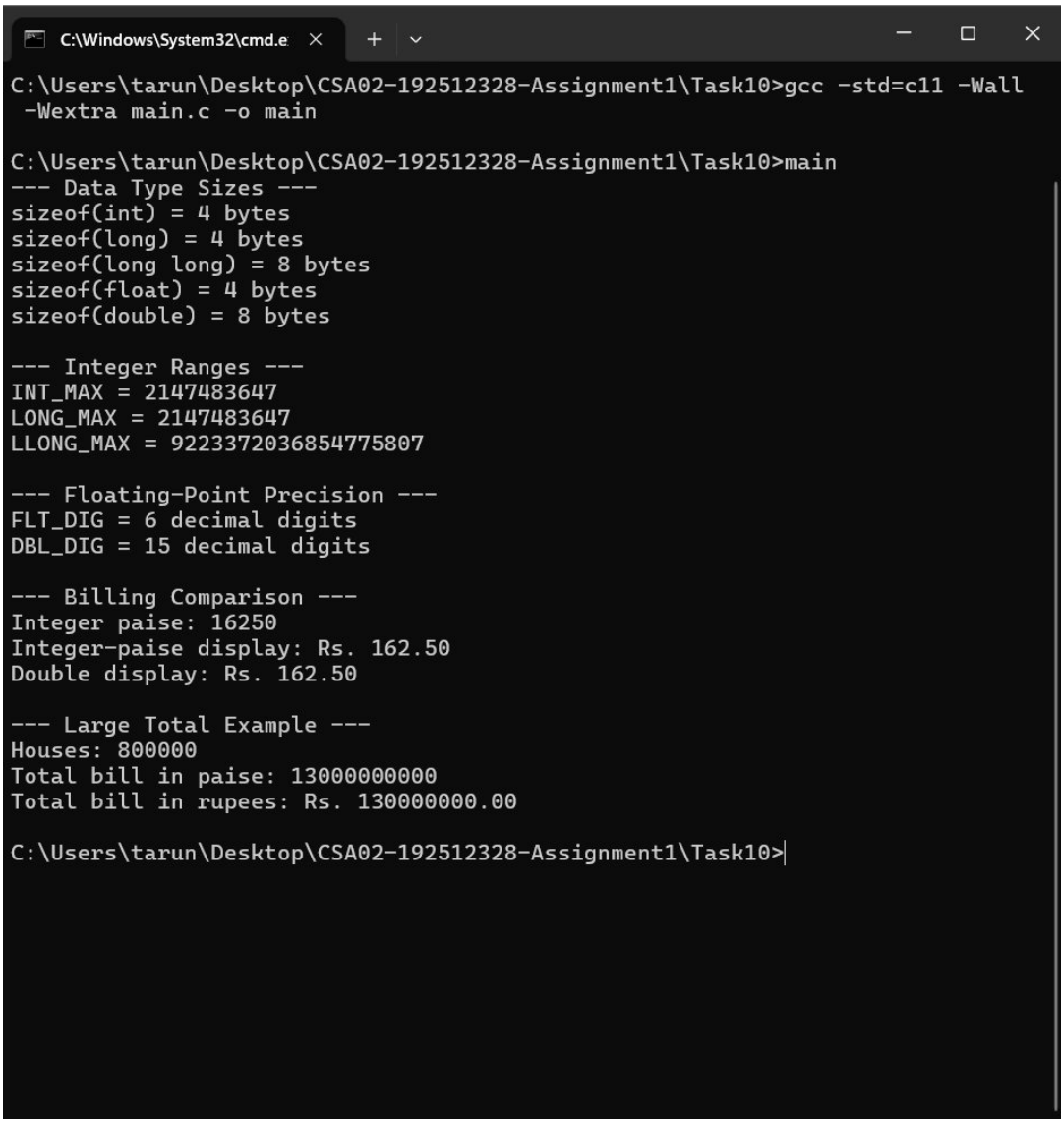
Task 10: Data types and large totals

main.c prints type sizes/ranges, precision indicators, and a large accumulated total.

```
#include <stdio.h>
#include <limits.h>
#include <float.h>
#include <stdint.h>
```

```
int main(void)
{
    printf("--- Data Type Sizes ---\n");
    printf("sizeof(int) = %zu bytes\n", sizeof(int));
    printf("sizeof(long) = %zu bytes\n", sizeof(long));
    printf("sizeof(long long) = %zu bytes\n", sizeof(long long));
    printf("sizeof(float) = %zu bytes\n", sizeof(float));
    printf("sizeof(double) = %zu bytes\n", sizeof(double));
... (see full file in Task10 folder)
```

Evidence screenshot for Task 10:



Task 11: Design recommendations

design_notes.txt discusses types, constants, integer paise, rollover, validation, testing, and improvements.

```
Task 11 – Final Design Recommendations

1. Data Types
```

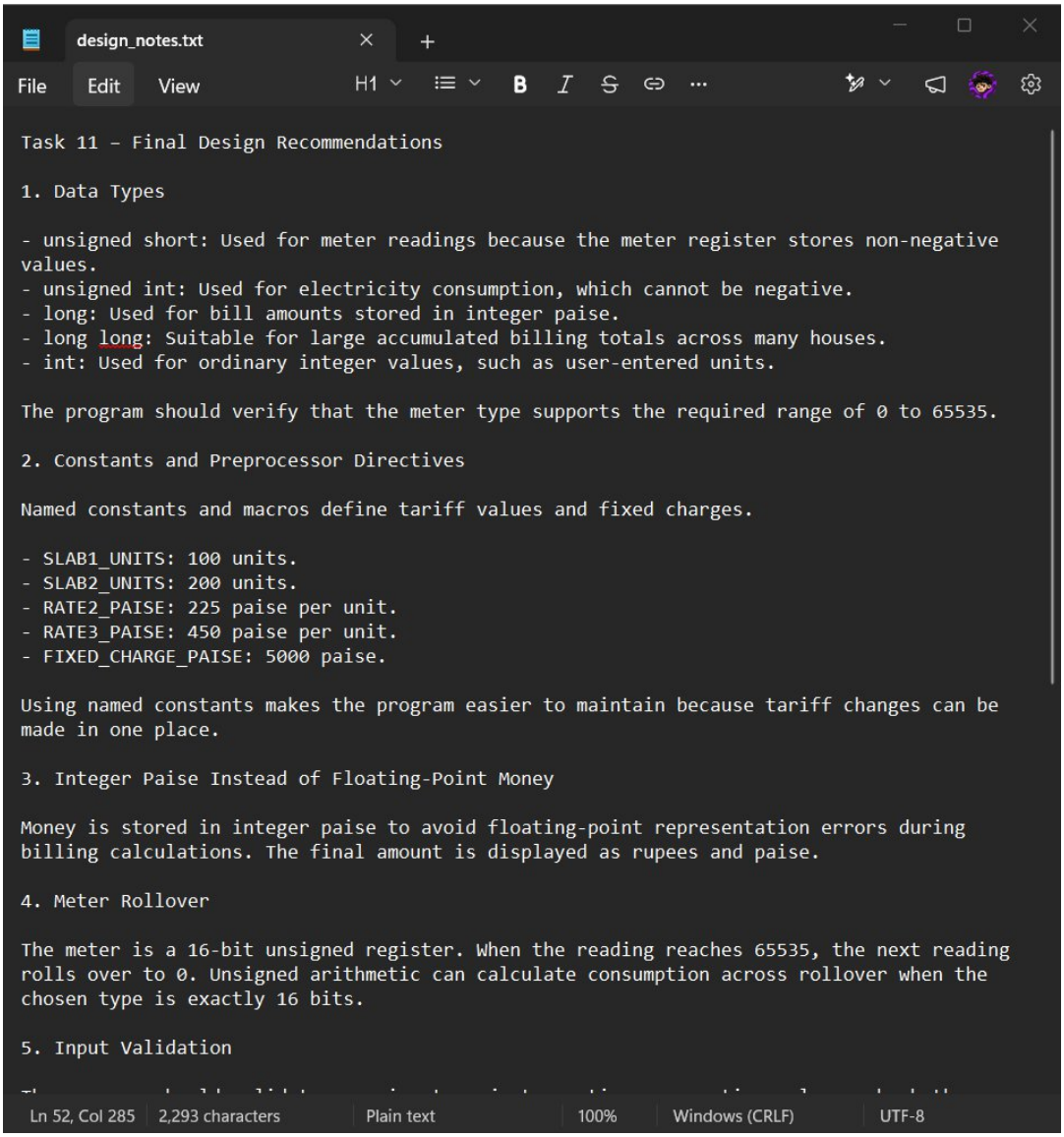
```
- unsigned short: Used for meter readings because the meter register stores non-negative values.
- unsigned int: Used for electricity consumption, which cannot be negative.
- long: Used for bill amounts stored in integer paise.
- long long: Suitable for large accumulated billing totals across many houses.
- int: Used for ordinary integer values, such as user-entered units.

The program should verify that the meter type supports the required range of 0 to 65535.

2. Constants and Preprocessor Directives

... (see full file in Task11 folder)
```

Evidence screenshot for Task 11:



Task 12: Smart meter design challenge

design_notes.txt describes 15-minute readings, peak tariff, runtime configuration, storage, validation, and tests.

```
Task 12 – Smart Meter Design Challenge
```

1. Problem Description

The smart meter system records electricity usage every 15 minutes. Each house produces 35,040 readings per year. The system must calculate electricity bills using different tariff rates depending on the time of day.

2. Time-of-Day Tariff

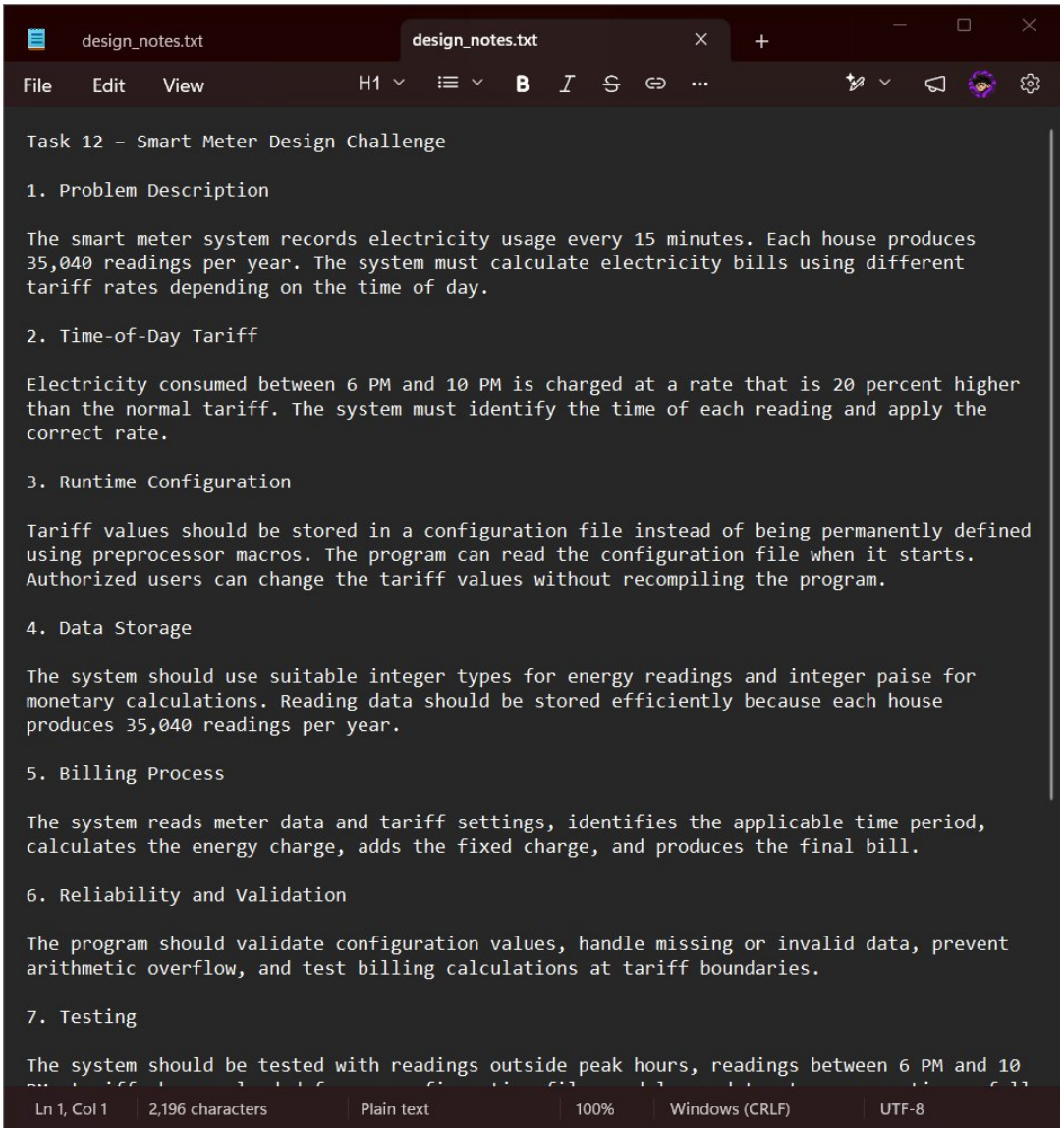
Electricity consumed between 6 PM and 10 PM is charged at a rate that is 20 percent higher than the normal tariff. The system must identify the time of each reading and apply the correct rate.

3. Runtime Configuration

Tariff values should be stored in a configuration file instead of being permanently defined using preprocessor macros. The program can read the configuration file when it starts. Authorized users can change the tariff values without recompiling the program.

... (see full file in Task12 folder)

Evidence screenshot for Task 12:



7. Test Plan and Results

The following expected results are calculated from the assignment tariff. “Expected” means formula-derived; the attached screenshots are the evidence files supplied in the project archive. Verify each actual run against these values before claiming a test passed.

Test	Input	Expected result
Normal meter use	Previous 1200; current 1530	330 units
Meter rollover	Previous 65400; current 120	256 units
Boundary rollover	Previous 65535; current 0	1 unit (16-bit register)
Zero units	0	₹50.00
First slab boundary	100	₹50.00
Second slab example	150	₹162.50
Second slab boundary	200	₹275.00
Third slab example	350	₹950.00
Negative units	-10	Reject with error
Non-numeric input	abc	Reject with error if tested by Task 06 program

7.1 Billing Calculation Workings

Units	Energy charge	Fixed charge	Total bill
0	₹0.00	₹50.00	₹50.00
100	₹0.00	₹50.00	₹50.00
150	50 × ₹2.25 = ₹112.50	₹50.00	₹162.50
200	100 × ₹2.25 = ₹225.00	₹50.00	₹275.00
350	₹225.00 + 150 × ₹4.50 = ₹900.00	₹50.00	₹950.00

Evidence note: the source archive includes output screenshots for Tasks 02–12. Task 01 is represented by analysis.txt. The ZIP does not include a separate invalid_input.png file, so do not claim a separate invalid-input screenshot unless you add one.

8. Compilation Stages

Stage	GCC command	Output
Preprocessing	gcc -E main.c -o main.i	Expanded source
Compilation to assembly	gcc -S main.i -o main.s	Assembly source
Assembly	gcc -c main.s -o main.o	Object file
Linking	gcc main.o -o main	Executable

The Task 08 folder in the uploaded archive contains main.i, main.s, main.o, main.exe, and output.png. This supports the compilation-stage demonstration.

9. Complexity Analysis

Operation / task	Time complexity	Auxiliary space	Reason
Task 01: analyze a single fixed-width reading	O(1)	O(1)	Fixed number of input fields and arithmetic operations.
Task 02 / Task 09: consumption calculation	O(1)	O(1)	Subtraction, comparison, and rollover arithmetic.
Task 03 / Task 06 / Task 09: one bill	O(1)	O(1)	Fixed number of slab comparisons and arithmetic operations.
Task 04: macro / inline demonstration	O(1)	O(1)	A fixed set of expressions is evaluated.
Task 05 / Task 07 / Task 10 demonstrations	O(1)	O(1)	Fixed number of demonstrations and printed values.
Task 08: compilation	Not expressed as O(1)	Depends on source/object size	Compiler complexity depends on program size and compiler implementation.
Process n readings once	O(n)	O(1) streaming auxiliary space	Each reading is processed a constant number of times.
Store n readings in memory	O(n) processing to ingest	O(n) storage	One record is retained per reading.
Smart meter annual readings per house	n = 35,040	O(n) if retained	15-minute intervals × 96 readings/day × 365 days.

For a streaming design, working memory can remain O(1) apart from the storage system, but retaining all annual records requires O(n) persistent storage.

10. Numeric Accuracy and Overflow

The program uses integer paise in the primary billing calculations: ₹50 = 5000 paise, ₹2.25 = 225 paise, and ₹4.50 = 450 paise. This avoids floating-point rounding during whole-paise calculations.

The C language does not guarantee that unsigned short is 16 bits on every platform. The meter implementation should verify the range and explicitly use a suitable fixed-width type or check USHRT_MAX == 65535. Arithmetic must also be checked for overflow for large user-entered unit counts and aggregated totals.

Task 10 demonstrates a large total: 800,000 houses × 16,250 paise = 13,000,000,000 paise = ₹130,000,000.00. This value fits in a sufficiently wide long long on common platforms, but range assumptions should be verified.

11. Smart Meter Design Challenge

Requirement	Design response
Reading interval	One reading every 15 minutes.
Annual count	96 readings/day × 365 days = 35,040 readings per house per year.
Peak window	6 PM–10 PM; proposed tariff is 20% above normal tariff.
Runtime tariffs	Read validated tariff values from a configuration file instead of compile-time-only macros.
Storage	Store meter identifier, timestamp, reading, and audit information.
Reliability	Validate missing/duplicate readings, tariff configuration, resets, and arithmetic limits.

The uploaded Task 12 is design notes rather than a complete implemented smart-meter system. Tariff treatment at interval boundaries and the precise conversion from meter readings to interval energy should be specified before production use.

12. Limitations and Future Improvements

The task programs are separate learning demonstrations, not a unified production application. The meter implementation assumes a 16-bit register and must verify that assumption. The basic billing program has no customer database, telemetry interface, or persistent billing ledger.

Future improvements include combining the demonstrations into a modular application, adding automated unit tests, strengthening input and overflow validation, separating declarations into header files, storing tariff configuration at runtime, and adding secure meter communication and audit logging.

13. Conclusion

This assignment demonstrates C programming techniques for electricity billing, including data types, integer-paise calculations, meter rollover, preprocessor directives, input validation, type conversion, compilation stages, testing, and complexity analysis. The included smart-meter proposal extends the basic scenario to time-of-day tariffs and 15-minute readings. The report distinguishes implemented task demonstrations from proposed future functionality.

14. References

• Assignment brief: Design, Implement and Validate a Reliable Billing Program for a Rural Electricity Co-operative.

• GCC documentation: compiler options and preprocessing/compilation stages.

• C standard library documentation for input/output functions and numeric limits.

• Course materials and lecture notes supplied for the C Programming Assignment 1.